

CodeGraph — Code Intelligence for Lava

CodeGraph is a local semantic code-knowledge-graph tool. It indexes the Lava codebase into a SQLite graph and exposes it to AI coding agents over the **Model Context Protocol (MCP)**, so an agent can resolve symbols, callers, callees, and change-impact instantly instead of repeatedly scanning files.

- Upstream: <https://github.com/colbymchenry/codegraph> · npm @colbymchenry/codegraph
- 100 % local — SQLite, no cloud, no external API (compatible with Lava's **Local-Only CI/CD** constitutional constraint).
- **Mandatory** for Lava development per the constitution (constitution/ — HelixConstitution codegraph mandate). All five supported CLI agents are wired.

Status: incorporated 2026-05-20. Design spec: docs/superpowers/specs/2026-05-20-codegraph-incorporation-design.md.

1. What gets created in the repo

Path	Tracked?	Purpose
.codegraph/ config.json	tracked	Include/exclude globs. Lava domain code + own-org submodule source in-scope AND physically indexed per §11.4.79 (blanket submodules/exclude removed); secrets + third-party + build artifacts excluded per §6.H / §11.4.79.
.codegraph/ codegraph.db	gitignored	The SQLite knowledge graph — a regenerable build artifact (CONST-053).
.codegraph/.gitignore		

Path	Tracked?	Purpose
	 tracked	codegraph's own ignore rules for its data files.
.mcp.json	 tracked	Claude Code project-scoped MCP server registration.
opencode.json	 tracked	OpenCode MCP server registration.
.crush.json	tracked	Crush MCP server registration.
.qwen/settings.json	tracked	Qwen Code MCP server registration.
~/.kimi/mcp.json	host-local	Kimi CLI MCP registration (Kimi config is host-local; .kimi/ is gitignored by project convention).

All MCP configs reference the binary as bare codegraph (resolved on PATH) — **never** an absolute host path, per the §6.R no-hardcoding mandate.

2. Installation

CodeGraph is a Node.js 18+ tool. Install it **globally** (no sudo — §6.U; the npm prefix is user-writable):

```
npm install -g @colbymchenry/codegraph
codegraph --version           # expect 0.6.x or newer
```

Native modules (better-sqlite3, tree-sitter) compile on install; codegraph ships a WASM fallback if a prebuilt binary is unavailable for your Node version.

3. Initialization & indexing

Already done for this repo — .codegraph/config.json is committed. To rebuild the index from scratch on a fresh clone:

```
codegraph index          # builds .codegraph/codegraph.db from
                           config.json
codegraph status         # shows file/node/edge counts – confirm
                           non-empty
```

config.json puts **Lava domain code AND own-org submodule SOURCE in-scope**, per constitution §11.4.79 (LVA-6) — which mandates that own-org (vasic-digital + HelixDevelopment) submodule source be indexed and forbids a blanket submodules/ exclude. Physically indexed today (verified 2026-05-31): app/, core/, feature/, buildSrc/, lava-api-go/, **plus the SOURCE of all 17 own-org functional submodules** under submodules/<name>/ (auth, cache, challenges, concurrency, config, containers, database, discovery, helixqa, http3, mdns, middleware, observability, ratelimiter, recovery, security, tracker_sdk). Totals: **3,167 files / 52,562 nodes / 137,980 edges**, of which **1,842 submodule files (1,673 .kt+.go source)**. The cross-submodule probe (DefaultPluginRegistry/PluginRegistry, symbols that exist only in submodules/tracker_sdk/) resolves — §11.4.79 step 4 PASSES.

codegraph v0.9.7 discovers submodule source via `git ls-files -c --recurse-submodules` (its `extraction/index.js` → `collectGitFiles`), which only lists submodules that are **git-active**. The **operational precondition** is therefore: before `codegraph index`, the `submodules/*` paths **MUST** be `git submodule init-active` (run `git submodule update --init`, respecting pins — **NEVER --remote**). A fresh clone with uninitialized submodules silently indexes 0 submodule files. The earlier "codegraph walker does not cross the gitlink boundary" claim was **INCORRECT** for v0.9.7 and is retracted — the real gate was submodule registration in `.git/config`, not a codegraph capability gap. Full record + falsifiability proof: `docs/codegraph-11479-reconciliation.md` → "Remediation attempt 2026-05-31".

Excludes (no blanket submodules/ entry — §11.4.79 clause 1): the constitution/ governance submodule (docs, not callable code — §11.4.79 clause 4), releases/, build/, .lava-ci-evidence/; nested third-party/vendored deps `**/vendor/`, `**/third_party/` (§11.4.79 clause 3); and — per §6.H / §11.4.10, with `**/` prefixes so they apply **INSIDE** submodules too — `.env*`, `keystores/`, `*.keystore`, `*.jks`, `google-services.json`, `firebase-admin-*.json`, `secrets/`. **Never remove the secret excludes**. The full reconciliation record (before/after policy, re-index + credential-leak verification) is in `docs/codegraph-11479-reconciliation.md`.

After changing the exclude policy you MUST `codegraph index` (single heavy op, §6.T.2), record the new file/node/edge counts, and run the credential-leak check (`codegraph files | grep -iE '\.env|keystore|\.jks|google-services|secrets/' | wc -l` → expect 0) per §11.4.79 step 5 + §11.4.78.

Keep the index fresh after edits:

```
codegraph sync           # incremental update since last index
```

4. The five supported CLI agents

Claude Code is Lava's primary agent; all five are wired and verified.

Agent	Config file	Re-add command
Claude Code	.mcp.json (project)	already present; <code>claude mcp get codegraph</code>
OpenCode	opencode.json (project)	already present; <code>opencode mcp list</code>
Qwen Code	.qwen/settings.json (project)	<code>qwen mcp add codegraph codegraph serve --mcp --scope project --transport stdio</code>
Kimi CLI	~/.kimi/mcp.json (host)	<code>kimi mcp add --transport stdio codegraph -- codegraph serve --mcp</code>
Crush	.crush.json (project)	already present

On a fresh clone the four project-scoped configs are committed and work immediately. Only **Kimi CLI** needs the one-off `kimi mcp add` above, because Kimi stores MCP config host-locally (`.kimi/` is gitignored).

All agents spawn `codegraph serve --mcp` (stdio transport) and gain the MCP tools: `codegraph_search`, `codegraph_context`, `codegraph_callers`, `codegraph_callees`, `codegraph_impact`, `codegraph_node`, `codegraph_files`, `codegraph_status`.

5. Verification — the anti-bluff suite

Per the Anti-Bluff Pact (§6.J / §6.L), the integration is covered by a falsifiable verification suite. It IS the codegraph quality gate (Local-Only CI/CD — no hosted equivalent).

```
scripts/verify-codegraph.sh          # full run – 6 layers,
    incl. agent E2E
scripts/verify-codegraph.sh --quick  # layers 01-04 + 06 (skip
    slow LLM layer)
```

Layer	Proves
01 index reality	codegraph indexed the real Lava codebase (file/node counts, Kotlin+Go) with no credential/secret-file-path leak (§6.H — .env/keystore/.jks/google-services.json/secrets/ <i>paths</i> absent; verified 0 on the 2026-05-31 re-index. Note: java.security.KeyStore / android.security.keystore.* <i>API identifiers</i> legitimately appear as imports in Lava source — those are code, not secret files). Own-org submodule source is in-scope per §11.4.79 AND physically indexed (1,842 submodule files / 1,673 .kt+.go), provided submodules are git submodule init-active before indexing — see docs/codegraph-11479-reconciliation.md → "Remediation attempt 2026-05-31".
02 query correctness	codegraph query resolves real symbols to real file:line.
03 MCP protocol	the MCP server returns real data over real stdio JSON-RPC.
04 agent connectivity	all 5 agents register/connect to codegraph.
05 agent E2E	each agent provably uses codegraph in a real LLM turn — it must report the index node count, a fact obtainable only by calling the codegraph_status tool.
06 falsifiability	layers 01-03 fail when the DB is removed and pass when restored — proving the suite is not a bluff.

Evidence is written to .lava-ci-evidence/codegraph/<UTC-timestamp>/.

Honest classification (§6.L): layer 05 marks an agent SKIP (a *documented gap*, never a faked pass) if it cannot be run here because it is not authenticated / has no model configured. Authenticate that agent and re-run. A FAIL means an agent ran but did not use codegraph — a real defect.

6. Troubleshooting

Symptom	Fix
codegraph: command not found	<code>npm install -g @colbymchenry/codegraph;</code> ensure the npm global bin is on PATH.
Agent does not see codegraph tools	re-check the agent's config file (§4); run the agent's <code>mcp list</code> .
codegraph status shows 0 files	run <code>codegraph index</code> ; check <code>config.json</code> include/exclude globs.
Stale results after edits	<code>codegraph sync</code> (or <code>codegraph index -f</code> for a full rebuild).
unlock needed	<code>codegraph unlock</code> removes a stale lock file blocking indexing.
codegraph status fails — "unable to open database file" / "Using WASM SQLite backend"	The native better-sqlite3 binding got disabled (a brew operation can disturb a global install placed under the Homebrew Node Cellar). Rebuild the index: <code>codegraph index</code> . If it persists, reinstall: <code>npm install -g @colbymchenry/codegraph.scripts/verify-codegraph.sh</code> pre-flight now detects this and aborts cleanly.
A secret path appeared in the index	a §6.H violation — add the path to <code>config.json</code> exclude and <code>codegraph index -f</code> ; file an incident.

7. Constitutional notes

- **§6.H** — `config.json` excludes every secret path (with `**/` prefixes so the excludes apply inside the now-indexed own-org submodules too); the index must never contain `.env`, `keystores`, `secrets/`, or Firebase keys.

- **§11.4.79 (LVA-6) — CLOSED 2026-05-31** — own-org submodule SOURCE (`vasic-digital + HelixDevelopment`) IS physically indexed (1,842 submodule files; cross-submodule probe passes); no blanket submodules/ exclude. Operational precondition: `git submodule init-active` before `codegraph index`. Third-party (`vendor/`, `third_party/`) and the `constitution/` `governance` submodule stay excluded. See `docs/codegraph-11479-reconciliation.md`.
- **§6.R** — MCP configs use the bare `codegraph` command (`PATH-resolved`); no hardcoded host paths.
- **§6.W** — `codegraph` is an npm tool; no Git remote is added.
- **Decoupled Reusable Architecture** — `codegraph` is third-party developer tooling, not Lava domain code and not a `vasic-digital` submodule.
- **Local-Only CI/CD** — `codegraph` and `scripts/verify-codegraph.sh` run entirely locally.